DHBW Heilbronn

Software Engineering II.1 – Block 3

Entwurf und Entwurfsmuster

Verantwortlichkeiten sichtbar und Änderungen beherrschbar machen

Software Engineering II.1 | WS 2026/27 | Dr. rer. nat. Ana Nicolaescu

Wo stehen wir im Kurs?

Block 1	Block 2	**Block 3**	Block 4	Block 5	Block 6	Block 7
4 UE	4 UE	4 UE	4 UE	4 UE	4 UE	4 UE
Einführung & Vorgehensmodelle	Requirements Engineering	Entwurf & Entwurfsmuster	Implementierung & Versionsmanagement	Testen (Blackbox)	Whitebox, Wartung & Change Request	Management & Abschlusspräsentation
Teams, Repo, Vorgehen	REQUIREMENTS.md Use Cases · AK · Prototyp	ARCHITECTURE.md Klassen- · Sequenzdiagramm	Build · Git · PR Review · CI grün	TESTING.md Unit/Integr. · Blackbox	Coverage · Refactoring Change Request	RISK_ANALYSIS.md Präsentation

Durchgängiges Gruppenprojekt „Photo Album“: jeder Block liefert ein prüfbares Artefakt ins Repository

Heute wird aus Anforderungen ein Entwurf

- Verantwortlichkeiten verteilen: Wo lebt eine Geschäftsregel?
- Kohäsion, Kopplung, Kapselung und Schnittstellen als Entscheidungskriterien
- Vom Use Case zu Klassen: Nomen-Analyse und CRC-Karten
- Sequenzdiagramme als Laufzeitprüfung des Entwurfs
- Entwurfsmuster nur für konkrete Probleme: Strategy, Factory, Observer, Adapter
- Agentensysteme entwerfen: Trennung von Business-Logik, KI, Tools und Daten
- Entscheidungen als Architecture Decision Record festhalten

Projektergebnis dieses Blocks
ARCHITECTURE.md: Architektur + Begründung
Komponenten, Verantwortlichkeiten, Schnittstellen
Klassendiagramm (Domäne + Anwendung)
Sequenzdiagramm „Foto hinzufügen“
Mindestens ein ADR (Architecture Decision Record)

Ablauf heute (240 Minuten)

00–10	Einstieg: Wo gehört die Regel hin?	**Übung**
10–45	Entwurfskonzepte: Schichten, Ports und Adapter, Kohäsion, Kopplung	**Übung**
45–75	Vom Use Case zu Klassen: Nomen-Analyse und CRC-Karten	**Übung**
75–100	Sequenzdiagramme: Notation und „Foto hinzufügen“	**Übung**
100–115	Pause	**Pause**
115–150	Entwurfsmuster: Strategy, Factory, Observer, Adapter – und ihre Risiken	**Übung**
150–190	KI-Perspektive: Agent als Schleife, Tool Calling im Code, Freigabe	**Übung**
190–215	Architecture Studio: Ihr Entwurf + ADR	**Teamarbeit**
215–235	Diagramm-Review über Kreuz · Projektcheckpoint	**Teamarbeit**
235–240	Exit Ticket · Hausaufgabe	

Einstieg: Wo gehört die Regel hin?

Übung

Geschäftsregel aus Block 2: „Ein Album darf höchstens 100 Fotos enthalten.“ Wo im Code wird sie geprüft?

6 min

Einzeln, dann Team

AUFTRAG

1. 1 Minute allein: Entscheiden Sie – (A) in der Oberfläche, (B) in einem Service/Use Case, (C) in der Klasse Album selbst, (D) in der Datenbank. Handzeichen.
2. 3 Minuten im Team: Was passiert bei jeder Variante, wenn morgen eine zweite Oberfläche (REST-API) dazukommt? Und wenn die Grenze auf 200 steigt?
3. Einigen Sie sich auf eine Antwort mit Begründung in einem Satz.

ERGEBNIS Eine Team-Antwort (Buchstabe + Satz).

DANACH IM PLENUM Abstimmung per Handzeichen, zwei Begründungen hören. Auflösung kommt nach dem Theorieteil – nicht jetzt.

1

Entwurfskonzepte kompakt

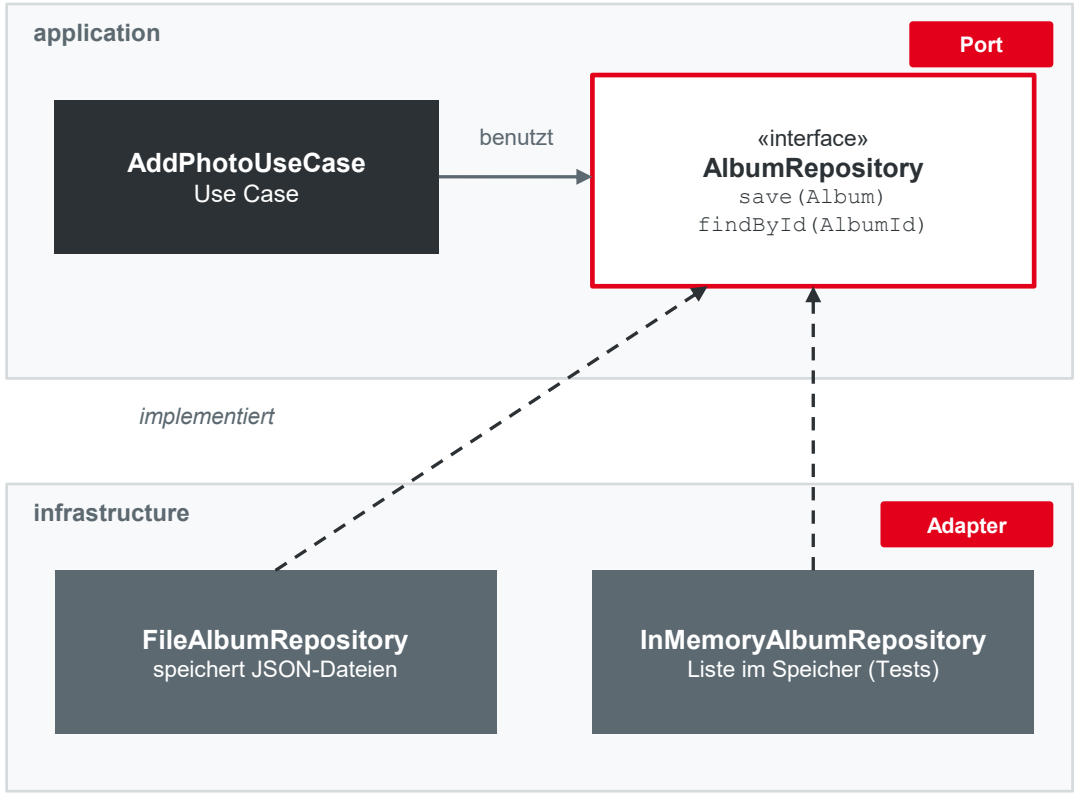
Zwölf Begriffe, vier Fragen, eine Architektur für das Photo Album.

Entwurf verteilt Verantwortung: Schichten des Photo Albums



Abhängigkeiten zeigen nach innen. Die Domäne kennt weder UI noch Datei noch KI. Genau dort wohnen die Geschäftsregeln aus Block 2.

Ports und Adapter: Die Anwendung sagt, was sie braucht



1 Port

Interface in der Anwendungsschicht. Beschreibt in Fachsprache, was der Use Case braucht: save(Album) – nicht writeJson(path).

2 Adapter

Klasse in der Infrastruktur. Implementiert den Port und übersetzt in Technik: Dateien, Bibliotheken, HTTP, KI-Dienst.

3 Verdrahtung

Genau eine Stelle (Main) entscheidet, welcher Adapter eingesteckt wird: im Test InMemory, im Betrieb File.

Die Abhängigkeit zeigt von der Technik zur Anwendung – nicht umgekehrt. Darum können Sie Technik tauschen, ohne einen Use Case anzufassen.

Ports und Adapter im Code: vier Stücke, eine Verdrahtung

① Port – Paket application

```
public interface AlbumRepository {
    Optional<Album> findById(AlbumId id);
    void save(Album album);
}
```

② Use Case – kennt nur den Port

```
public class AddPhotoUseCase {
    private final AlbumRepository albums;

    public AddPhotoUseCase(AlbumRepository albums) {
        this.albums = albums;    // welcher? egal
    }
    public void execute(AlbumId id, Photo photo) {
        Album a = albums.findById(id).orElseThrow();
        a.addPhoto(photo);       // Regel: Domäne
        albums.save(a);
    }
}
```

③ Adapter – Paket infrastructure

```
public class FileAlbumRepository
        implements AlbumRepository {
    public void save(Album album) {
        String json = toJson(album);  // Technik
        Files.writeString(fileFor(album), json);
    }
    public Optional<Album> findById(AlbumId id) {...}
}
```

④ Verdrahtung – Main bzw. Test

```
// Main (Betrieb)
new AddPhotoUseCase(new FileAlbumRepository(dir));
// Test (Block 5)
new AddPhotoUseCase(new InMemoryAlbumRepository());
```

Unit-Test ohne Dateisystem	Speicher austauschbar: Datei → DB	Domäne frei von Technik

Merksatz: Der Use Case kennt nur das Interface. „new FileAlbumRepository(…)“ steht an genau einer Stelle – in Main.

Entwurfsziele entstehen aus Ihren nichtfunktionalen Anforderungen

NFA aus Block 2	Entwurfskonsequenz	Im Photo Album konkret
Änderbarkeit / Wartbarkeit	Modularisierung, stabile Schnittstellen, Regeln an einem Ort	Geschäftsregeln nur in der Domäne; Speicher hinter Repository-Interface
Testbarkeit	Abhängigkeiten von außen setzbar (Dependency Injection), keine statischen Zugriffe	AddPhotoUseCase(AlbumRepository repo, ImageHasher hasher) – im Test InMemory
Performance	kann zusätzliche Kopplung oder Caching rechtfertigen – bewusst entscheiden	Vorschaubilder cachen, statt jedes Mal 20-MB-Dateien zu laden
Sicherheit / Datenschutz	klare Vertrauensgrenzen, Validierung an der Grenze, minimale Rechte	Metadaten-Stripping vor KI-Aufruf im Adapter, nicht in der UI
Zuverlässigkeit	Fehler als Teil des Vertrags, keine Teilzustände	addPhoto ist atomar: erst prüfen, dann speichern, dann Album ändern

Entwurfskonzepte auf einen Blick

Konzept	Leitfrage	Beispiel Photo Album
Abstraktion	Was ist für diese Aufgabe relevant, was nicht?	Photo hat id, hash, titel – nicht die Pixel
Modularisierung / Dekomposition	In welche Teile mit eigenem Zweck zerlege ich?	Pakete domain, application, infrastructure, ui
Kapselung	Welche Daten dürfen nur kontrolliert verändert werden?	Album.photos ist privat; nur addPhoto/removePhoto ändern sie
Schnittstellen	Welchen Vertrag biete ich, ohne die Umsetzung zu zeigen?	interface AlbumRepository { save, findById, findAll }
Verantwortlichkeiten	Wer entscheidet was? (eine Klasse, eine Aufgabe)	Album prüft Regeln; UseCase koordiniert; UI zeigt
Kohäsion	Gehört alles in dieser Einheit zusammen?	ImageHasher berechnet nur Hashes, keine Dateiverwaltung
Kopplung	Wie viele andere Teile muss ich kennen und ändern?	UI kennt UseCase-Interface, nicht das Repository
Wiederverwendbarkeit / Erweiterbarkeit	Kann ich es woanders nutzen / ergänzen ohne Umbau?	Neue Sortierung = neue Strategy-Klasse, kein if-Umbau
Testbarkeit	Kann ich es isoliert und schnell prüfen?	Domäne ohne Datei und UI testbar

Kohäsion hoch, Kopplung niedrig – die zwei Hebel

Hohe Kohäsion

- ???

Geringe Kopplung

- ???

Kohäsion und Kopplung sind die zwei Kriterien, mit denen Sie im Architecture Studio jede Zuordnung begründen. „Weil es sich richtig anfühlt“ zählt nicht.

Kohäsion hoch, Kopplung niedrig – die zwei Hebel

Hohe Kohäsion

- Eine Einheit, eine klar benennbare Aufgabe
- Daten und die Methoden, die sie verändern, liegen zusammen
- Warnsignal: Klassenname mit „Manager“, „Helper“, „Util“; Methoden, die verschiedene Felder nie gemeinsam nutzen
- Photo Album: Album hält Fotos UND prüft die 100er-Regel – das gehört zusammen

Geringe Kopplung

- Wenige, stabile Abhängigkeiten – am besten auf Schnittstellen
- Änderung in A erzwingt keine Änderung in B
- Warnsignal: import java.io.File in der UI; Klassen, die sich gegenseitig kennen (Zyklen)
- Photo Album: UseCase kennt AlbumRepository (Interface), nicht FileAlbumRepository

Kohäsion und Kopplung sind die zwei Kriterien, mit denen Sie im Architecture Studio jede Zuordnung begründen. „Weil es sich richtig anfühlt“ zählt nicht.

Kapselung schützt Invarianten – Beispiel Album.addPhoto

```
public class Album {
    public static final int MAX_PHOTOS = 100;
    private final String name;
    private final List<Photo> photos = new ArrayList<>();

    public Album(String name) {
        if (name == null || name.isBlank() || name.length() > 50)
            throw new InvalidAlbumNameException(name);
        this.name = name;
    }

    public void addPhoto(Photo photo) {
        if (photos.size() >= MAX_PHOTOS)
            throw new AlbumFullException(name, MAX_PHOTOS);
        boolean duplicate = photos.stream()
            .anyMatch(p -> p.hash().equals(photo.hash()));
        if (duplicate) throw new DuplicatePhotoException(photo);
        photos.add(photo);
    }

    public List<Photo> photos() { return List.copyOf(photos); }
}
```

- Feld photos ist privat – kein setPhotos(), keine Referenz nach außen (copyOf)
- Invarianten (FA-01, FA-05, Regel 100) sind an genau einer Stelle prüfbar
- Fehler sind fachliche Exceptions, keine booleschen Rückgabewerte – der Use Case übersetzt sie für die UI
- Abstraktion: Photo kennt seinen Hash, nicht die Bytes
- Diese Klasse ist ohne UI, Datei und Framework testbar – Block 5

Vererbung oder Benutzung?

Vererbung (is-a) – sparsam

- ???

Benutzung (has-a / uses) – Standard

- ???

Vererbung oder Benutzung?

Vererbung (is-a) – sparsam

- Nur bei echter Ist-ein-Beziehung, bei der die Unterklasse überall für die Oberklasse einstehen kann (Substitutionsprinzip)
- Warnsignale: Vererbung nur, um Code zu sparen; Unterklasse wirft in geerbten Methoden UnsupportedOperationException
- Photo Album: braucht man Vererbung? Vermutlich nicht – ein Album ist kein Foto, ein Foto keine Datei

Benutzung (has-a / uses) – Standard

- Komposition und Delegation: Album hat Fotos; AddPhotoUseCase benutzt Repository und Hasher
- Leichter testbar (Abhängigkeit austauschbar), keine versteckte Kopplung an Oberklassen-Internas
- Schnittstelle statt abstrakter Klasse, wenn nur der Vertrag geteilt wird
- Regel aus 02_Inheritance: „Favor composition over inheritance“

Übung: Kopplungs-Check

Übung

Vier Aussagen über einen Entwurf. Welche erhöhen die Kopplung unnötig, welche sind in Ordnung – und warum?

5 min

Paare

AUFTRAG

1. Die JavaFX-Oberfläche ruft direkt FileAlbumRepository.save() auf.
2. AddPhotoUseCase bekommt AlbumRepository und ImageHasher im Konstruktor übergeben.
3. Album importiert javax.imageio, um beim Hinzufügen die Bildgröße zu prüfen.
4. Photo speichert den Pfad der Datei und liest beim Anzeigen selbst vom Dateisystem.

ERGEBNIS Pro Aussage: ok / problematisch + betroffenes Konzept (Kopplung, Kohäsion, Kapselung, Schicht).

2

Vom Use Case zu Klassen

Nomen-Analyse, CRC-Karten, Klassendiagramm – aus SAE bekannt, hier für Anwendung und Domäne erweitert.

Nomen-Analyse: vom Text zu Kandidaten

1	2	3	4	5
Nomen sammeln	**Aussortieren**	**Attribut oder Klasse?**	**Beziehungen**	**Verantwortung zuordnen**
aus Use Cases und Anforderungen: Album, Foto, Nutzerin, Titel, Datei, Hash, Fehlermeldung, Vorschlag …	irrelevant (Fehlermeldung = UI), Instanzen (‚Urlaub 2026‘), Synonyme (Bild = Foto)	Titel, Hash → Attribute von Foto. Beschreibung → Attribut. Album → Klasse	Verben: Album enthält Fotos (1..100), Nutzerin besitzt Alben	Wer prüft die 100er-Regel? Wer berechnet den Hash? → CRC-Karten

Aus 05a_Domain_Modeling_Example (Calendar Tool) übertragen. Ergebnis ist eine Hypothese – das Sequenzdiagramm prüft sie.

CRC-Karten: Klasse, Verantwortlichkeiten, Zusammenarbeit

Album

Verantwortlichkeiten	Zusammenarbeit
• kennt Name und Fotos • prüft: max. 100, kein Duplikat, Name 1–50 • fügt Foto hinzu / entfernt • benennt sich um	• Photo

AddPhotoUseCase

Verantwortlichkeiten	Zusammenarbeit
• lädt Album • lässt Datei prüfen und hashen • ruft album.addPhoto • speichert; übersetzt Fehler	• AlbumRepository • ImageHasher • Album

ImageHasher

Verantwortlichkeiten	Zusammenarbeit
• berechnet Inhalts-Hash einer Bilddatei • prüft Dateityp und Größe	• Dateisystem (Port)

AlbumRepository «interface»

Verantwortlichkeiten	Zusammenarbeit
• save(Album) • findById(id) • findAll() • delete(id)	• – (wird von Infrastruktur implementiert)

Faustregel: mehr als 5 Verantwortlichkeiten oder mehr als 3 Mitarbeiter → Karte teilen.

Übung: CRC-Karten für Ihr Photo Album

Übung

Nehmen Sie Ihre zwei Use Cases aus REQUIREMENTS.md und Ihre Geschäftsregeln.

15 min

Team

AUFTRAG

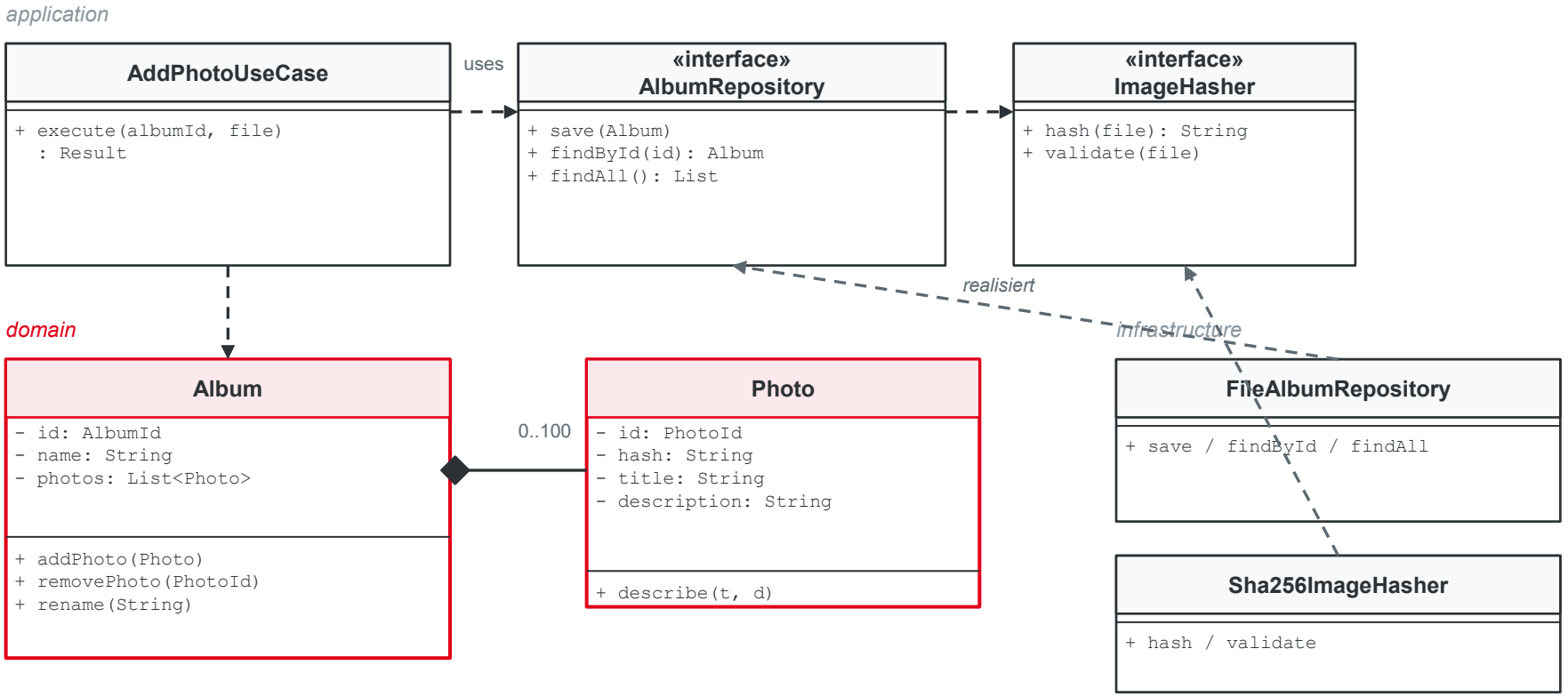
1. Nomen-Analyse (5 min): Kandidatenliste, Attribute abtrennen, Synonyme streichen.
2. CRC-Karten (8 min): eine Karte je Klasse auf Papier. Domäne, Anwendung (Use Cases), Ports (Interfaces). Jede Geschäftsregel steht auf genau einer Karte.
3. Walkthrough (2 min): Spielen Sie „Foto hinzufügen“ durch – jede Karte wird von einer Person gehalten, wer angesprochen wird, hebt sie. Bleibt eine Karte stumm? Übernimmt eine Karte alles?

ERGEBNIS Fotografierte CRC-Karten in docs/design/ (Rohmaterial für das Klassendiagramm).

MATERIAL Karteikarten (ich bringe 10 je Team mit), Stifte

DANACH IM PLENUM Ein Team zeigt seine Karte mit den meisten Einträgen – wir prüfen Kohäsion.

Klassendiagramm als Strukturhypothese



Abhängigkeitsrichtung: application → domain; infrastructure → application (implementiert Ports). Die Domäne zeigt nirgendwohin.

Vier Entwurfsfragen reichen für den Start

- **Wer kennt die fachliche Information?**
  - → Information Expert: Album kennt seine Fotos, also zählt Album
- **Wer entscheidet über die Regel?**
  - → die Klasse, deren Invariante es ist (Album), nie die UI
- **Wer koordiniert den Use Case?**
  - → genau eine Anwendungsklasse je Use Case (Controller); UI ruft sie nur auf
- **Welche Abhängigkeit soll austauschbar bleiben?**
  - → dafür ein Interface (Port): Speicher, Hashing, KI-Dienst, Uhrzeit

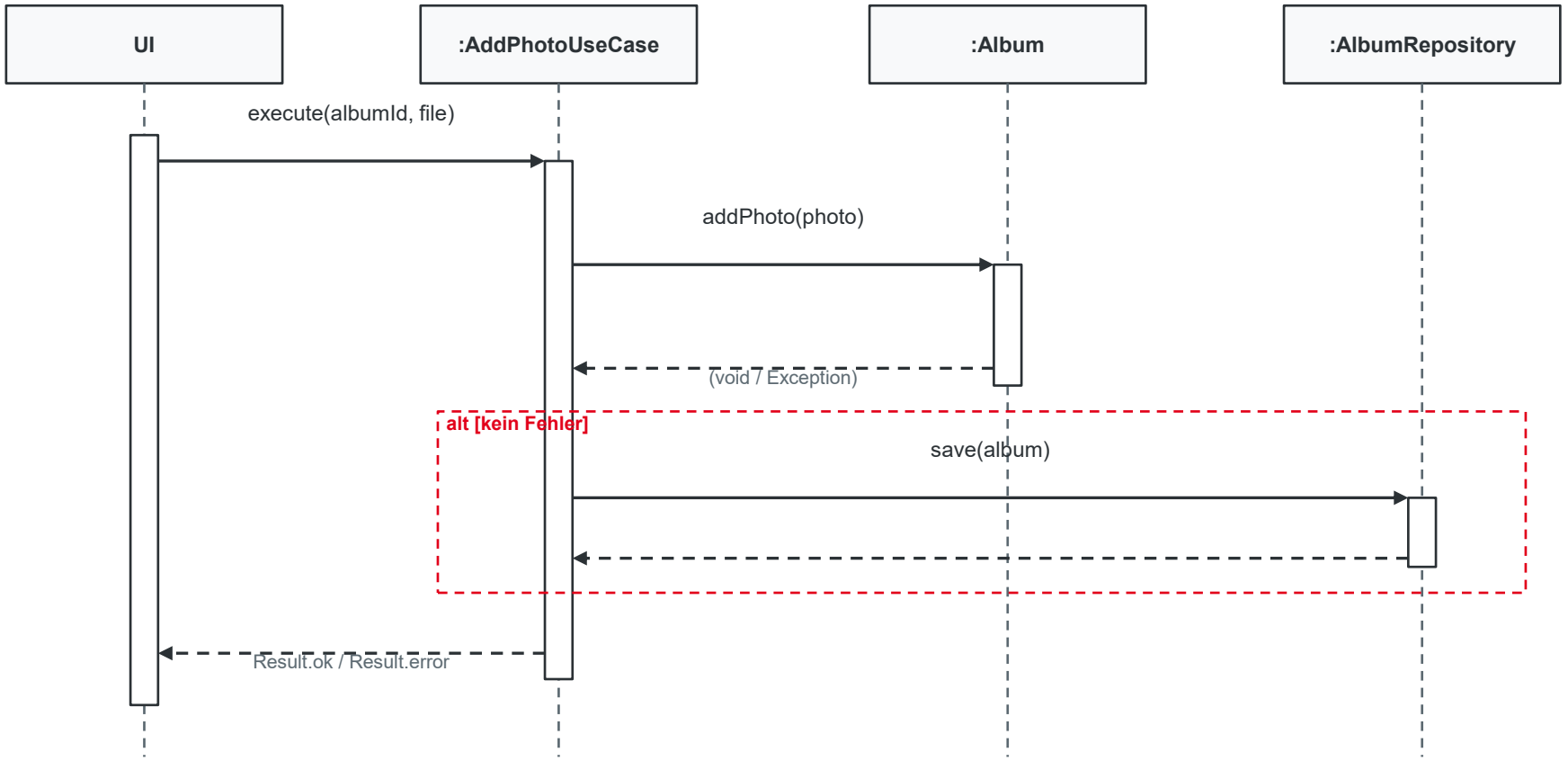
Das sind vier der GRASP-Prinzipien (Larman: Applying UML and Patterns – aus der SAE-Literaturliste). Sie beantworten 80 % der Zuordnungsfragen im Architecture Studio.

3

Sequenzdiagramme

Neu: Das Laufzeitverhalten prüft, ob die Verantwortungsverteilung funktioniert.

Sequenzdiagramm: Notation in fünf Elementen



Lebenslinie: Objekt/Rolle, Zeit läuft nach unten · Aktivierung: Objekt führt gerade aus · Synchrone Nachricht: Methodenaufruf, wartet auf Antwort · Antwort (gestrichelt): Rückgabewert oder Exception · alt / opt / loop: Bedingung, Option, Schleife

Sequenz für „Foto hinzufügen“: wer tut was?

ENTWURF A: UI MACHT ALLES

UI liest die Datei, berechnet den Hash, prüft in einer Schleife über alle Fotos auf Duplikate, prüft die Anzahl, schreibt die Datei und ändert die Fotoliste des Albums direkt (album.getPhotos().add(...)).

ENTWURF B: VERANTWORTUNG VERTEILT

UI ruft AddPhotoUseCase.execute(albumId, file) auf. Der Use Case lässt ImageHasher validieren und hashen, lädt das Album über AlbumRepository, ruft album.addPhoto(photo) – das Album prüft Kapazität und Duplikat – und speichert erst danach. Fehler kommen als fachliche Exceptions zurück und werden vom Use Case in ein Result für die UI übersetzt.

Entwurf A: 1 Objekt, 0 Testbarkeit ohne UI, jede zweite Oberfläche kopiert die Regeln. Entwurf B: jeder Pfeil im Sequenzdiagramm entspricht einer Zeile im Klassendiagramm – das ist die Konsistenzprüfung.

Übung: Sequenzdiagramm „Album löschen“

Übung

Auftraggeber-Entscheidung aus Block 2: Löschen eines Albums fragt nach und löscht die enthaltenen Fotos mit.

10 min

Team

AUFTRAG

1. Zeichnen Sie das Sequenzdiagramm für den Standardablauf mit UI, DeleteAlbumUseCase, Album, AlbumRepository, PhotoStorage (Port für Dateien).
2. Ergänzen Sie ein alt-Fragment: Nutzerin bricht die Bestätigung ab.
3. Prüfen Sie gegen Ihr CRC-Modell: Jede Nachricht = eine Verantwortlichkeit auf einer Karte? Wenn nicht: Karte oder Diagramm ändern.

DANACH IM PLENUM Ein Team zeigt sein Diagramm; Frage ans Plenum: Wer entscheidet über die Bestätigung – UI oder Use Case?

4

Entwurfsmuster

Bewährte Lösungen für wiederkehrende Probleme – und die Frage, ob Sie das Problem überhaupt haben.

Patterns lösen wiederkehrende Entwurfsprobleme

- Ein Entwurfsmuster beschreibt ein Problem, seinen Kontext, eine Lösung und ihre Konsequenzen – nicht nur eine Klassenstruktur (Gamma, Helm, Johnson, Vlissides 1994: „Gang of Four“)
- Drei Kategorien: **Erzeugung** (Factory, Singleton, Builder), **Struktur** (Adapter, Composite, Facade), **Verhalten** (Strategy, Observer, Template Method, Iterator)
- Nutzen: gemeinsames Vokabular („das ist ein Adapter“), erprobte Konsequenzen, dokumentierte Trade-offs
- Gefahr: Pattern als Selbstzweck. Projektbeschreibung: „Der Einsatz eines Patterns allein zur Erhöhung der Anzahl wird nicht positiv bewertet.“
- Heute vier Muster, die im Photo Album ein echtes Problem lösen können – und für jedes die Alternative ohne Pattern

Pattern-Auswahl: erst das Problem, dann der Name



Was wird sich wahrscheinlich ändern? (Speicherort, Sortierung, UI-Technik, KI-Anbieter)

„Beim Wechsel von Datei auf Datenbank müssten wir 7 Klassen anfassen.“

if/else, Kopie, direkte Abhängigkeit – wie teuer ist das wirklich?

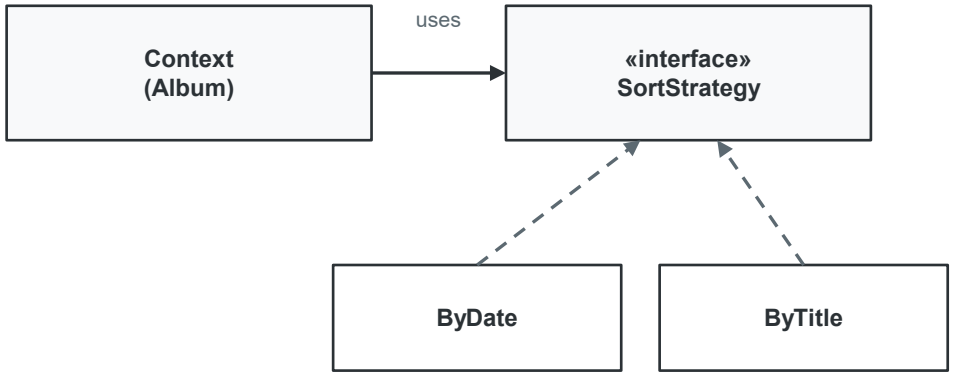
mehr Klassen, mehr Indirektion, dafür austauschbar und testbar

Kontext, Entscheidung, Alternativen, Konsequenzen – 10 Zeilen

Regel für das Projekt: Ein Pattern ohne ADR ist ein Verdachtsfall.

Strategy: austauschbares Verhalten zur Laufzeit

Struktur



Problem: Verhalten variiert, Auswahl zur Laufzeit.
Lösung: Verhalten in eigene Klassen kapseln, gemeinsames Interface, Context delegiert.
Konsequenzen: + offen für neue Varianten, + testbar je Variante; – mehr Klassen, Client muss Strategie kennen/wählen.

Photo Album

```
interface SortStrategy {
  List<Photo> sort(List<Photo> photos);
}
class ByDate implements SortStrategy { ... }
class ByTitle implements SortStrategy { ... }

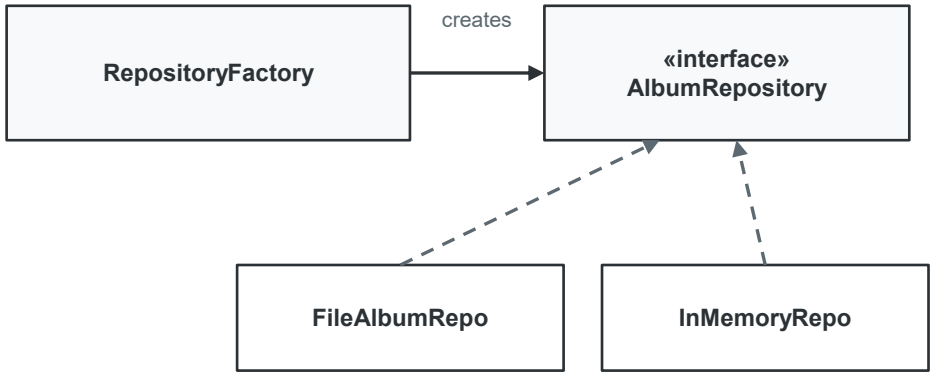
// Album (Context)
public List<Photo> photosSorted(SortStrategy s) {
  return s.sort(photos());
}

// Alternative ohne Pattern:
// switch (mode) { case DATE -> ...; case TITLE -> ...; }
// ok bei 2 festen Varianten – nicht bei wachsender Menge
```

Entscheidungsfrage: Erwarten wir mehr als zwei Varianten oder Auswahl durch die Nutzerin? Ja → Strategy. Nein → Comparator reicht.

Factory Method: Objekterzeugung kapseln

Struktur



Problem: Wer entscheidet, welche Implementierung erzeugt wird?
Lösung: Erzeugung hinter einer Methode/Klasse; Aufrufer arbeitet nur mit dem Interface.
Konsequenzen: + eine Stelle für Konfiguration, + Tests wählen ihre Implementierung; – zusätzliche Klasse, bei einer Implementierung unnötig.

Photo Album

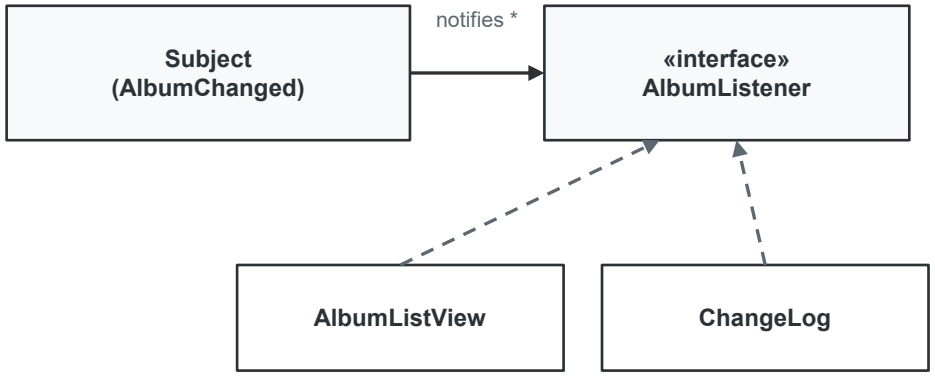
```
class RepositoryFactory {
  static AlbumRepository create(Config cfg) {
    return switch (cfg.storage()) {
      case FILE   -> new FileAlbumRepository(cfg.dir());
      case MEMORY -> new InMemoryAlbumRepository();
    };
  }
}

// Composition Root (main):
var repo   = RepositoryFactory.create(config);
var hasher = new Sha256ImageHasher();
var addPhoto = new AddPhotoUseCase(repo, hasher);
var ui = new AlbumWindow(addPhoto, ...);
```

Für das Projekt oft ausreichend: alle „new“ in einer main-Klasse (Composition Root). Factory erst, wenn die Auswahl konfigurierbar sein muss.

Observer: Änderungen verteilen, ohne die Empfänger zu kennen

Struktur



Problem: 1 Änderung, n Interessenten, die das Modell nicht kennen soll.
Lösung: Interessenten registrieren sich; Subject ruft sie bei Änderung auf.
Konsequenzen: + lose Kopplung Modell ↔ UI; – versteckter Kontrollfluss, Reihenfolge unklar, Memory Leaks bei vergessenem Abmelden.

Photo Album

```
interface AlbumListener { void onAlbumChanged(AlbumId id); }

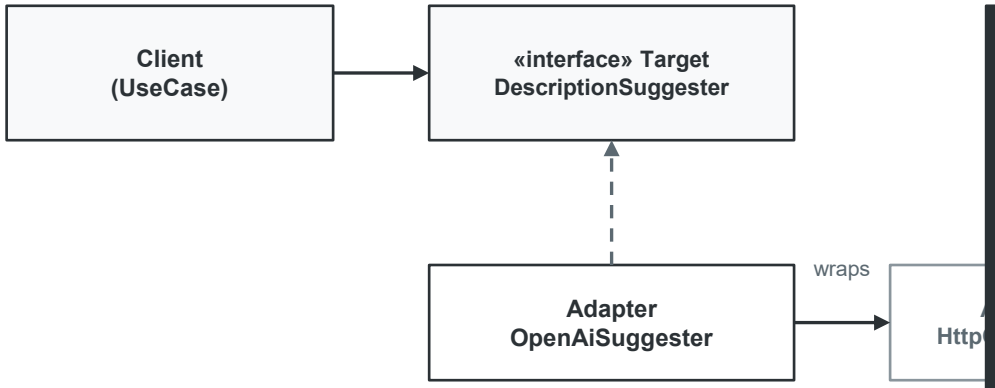
class AlbumEvents {
  private final List<AlbumListener> listeners = ...;
  void subscribe(AlbumListener l) { listeners.add(l); }
  void publish(AlbumId id) {
    listeners.forEach(l -> l.onAlbumChanged(id));
  }
}
// AddPhotoUseCase: nach save → events.publish(album.id())
// UI: events.subscribe(id -> listView.refresh());

// Alternative ohne Pattern: UseCase ruft ui.refresh()
// → Anwendung kennt UI, Schichtregel verletzt
```

Entscheidungsfrage: Muss die Anwendungsschicht die UI kennen? Nein → Observer (oder JavaFX-Properties). Bei einer einzigen Ansicht reicht ein Rückgabewert.

Adapter: fremde Schnittstellen übersetzen

Struktur



Problem: Schnittstelle des Fremdsystems passt nicht zu unserem Port.
Lösung: Adapter implementiert unser Interface und delegiert an das Fremdsystem.
Konsequenzen: + Fremdsystem austauschbar, + Tests mit Fake-Adapter, + zentrale Stelle für Fehler/Timeout/Guardrails; – eine Klasse mehr.

Photo Album

```
// Port (application)
interface DescriptionSuggester {
  Optional<String> suggest(Path image);
}

// Adapter (infrastructure)
class OpenAiSuggester implements DescriptionSuggester {
  public Optional<String> suggest(Path image) {
    byte[] stripped = Exif.strip(image);    // Guardrail 1
    var resp = client.describe(stripped, TIMEOUT_5S);
    return resp.map(Output::sanitize)       // Guardrail 2
               .filter(t -> t.length() <= 100);
  }
}
// Test: class FakeSuggester implements DescriptionSuggester
```

Adapter ist im Projekt fast immer gerechtfertigt: für Dateisystem, Hashing-Bibliothek, KI-Dienst. Er ist die Grenze zwischen Ihrem Code und der Welt.

Zwei weitere, die oft passen: Template Method und Repository

Template Method

- Fester Ablauf in der Oberklasse, variable Schritte als abstrakte Methoden
- Photo Album: Import-Ablauf (validieren → hashen → speichern) mit variablem Schritt „Vorschaubild erzeugen“ je Format
- Alternative: Strategy für den variablen Schritt – bevorzugt, wenn Vererbung sonst nicht nötig ist

Repository (kein GoF-Pattern, aber Standard)

- Sammlung fachlicher Objekte hinter einer Schnittstelle; versteckt, ob Datei, Datenbank oder Speicher
- Photo Album: AlbumRepository mit save/findById/findAll/delete – Sie haben es schon gezeichnet
- Regel: Das Repository enthält keine Geschäftsregeln – nur Speichern und Finden

Singleton ist für das Projekt fast nie die richtige Antwort: versteckte globale Abhängigkeit, schlecht testbar. Composition Root statt Singleton.

Pattern-Risiken: wann ein Muster schadet

- Zu frühe Abstraktion: ein Interface mit genau einer Implementierung, die sich nie ändert – erhöht nur die Leselast
- Zu viele Indirektionen: Factory erzeugt Strategy, die Adapter benutzt – niemand findet den eigentlichen Code
- Observer: versteckter Kontrollfluss, unklare Reihenfolge, vergessene Abmeldung (Memory Leak)
- Singleton: globale Zustände, Tests beeinflussen sich gegenseitig
- Pattern-Sprache ohne Pattern-Problem: „AbstractAlbumFactoryProvider“ für eine Klasse mit einem Konstruktor
- Faustregel: Ein Pattern rechtfertigt sich durch eine konkrete, wahrscheinliche Änderung – benennen Sie sie im ADR

Bewertungskriterium „Wesentliche Entwurfsentscheidungen begründet“ (5 Punkte): Für jedes Pattern die Alternative nennen, die Sie verworfen haben.

Pattern-Gericht

Übung

Anklage: „Das Team hat für den Speicher ein AlbumRepository-Interface mit Factory eingeführt, obwohl nur Dateispeicherung gefordert ist. Das ist unnötige Komplexität.“

10 min

Zwei Teams gegeneinander

AUFTRAG

1. Teams 1, 3, 5 sind Verteidigung, Teams 2, 4, 6 Anklage, Team 7 ist die Jury. 4 Minuten Vorbereitung: je zwei Argumente mit konkretem Änderungsszenario.
2. Je 90 Sekunden Plädoyer (eine Person pro Seite, Jury wählt).
3. Jury urteilt: Unter welcher Bedingung ist das Pattern gerechtfertigt, unter welcher nicht? Das Urteil wird als Muster-ADR für alle notiert.

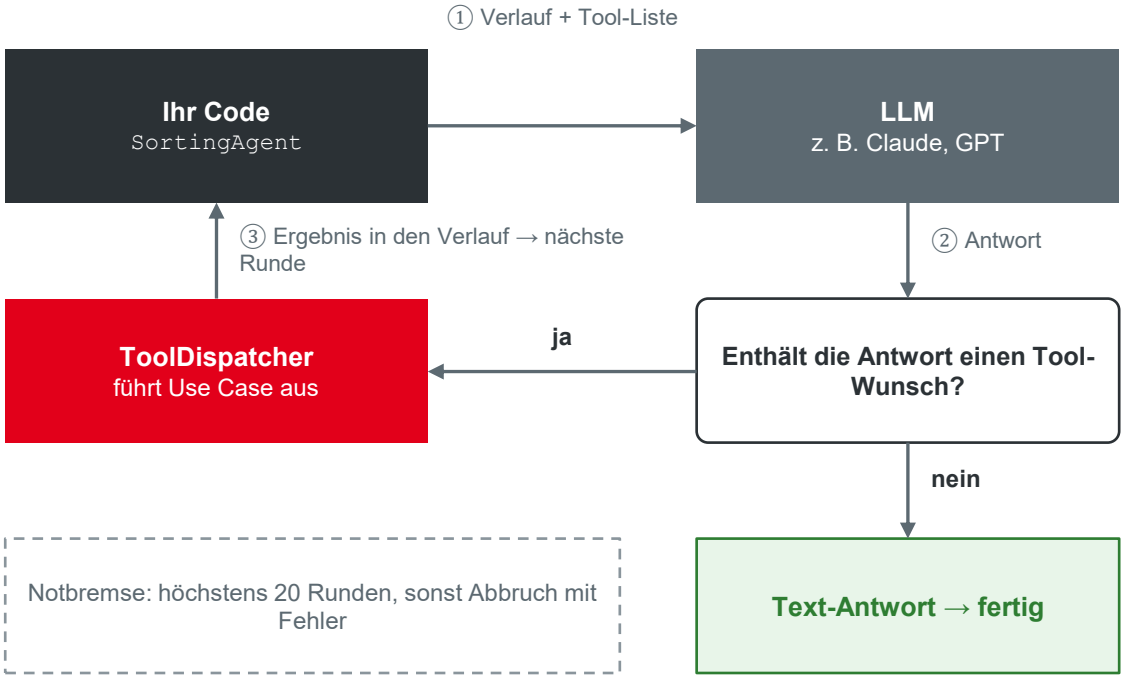
ERGEBNIS Ein Muster-ADR „Repository-Interface: ja/nein, weil …“, das alle Teams als Vorlage nutzen dürfen.

5

KI-Perspektive

Ein Agent ist eine Schleife in Ihrem Code: LLM, Tool-Liste, Dispatcher – und Grenzen, die im Code stehen.

Ein Agent ist eine Schleife in Ihrem Code – das LLM schreibt nur Text



Ein LLM-Aufruf = ein Schritt

Die Aufgabe braucht viele Schritte – deshalb die Schleife.

Nur zwei Antwortformen

Tool-Wunsch wie movePhoto("p7", "a3") oder Text. Mehr kann das LLM nicht.

Ausführen tut nur Ihr Code

Was Ihr Code nicht anbietet und nicht ausführt, kann der Agent nicht tun.

Agent = LLM + Tool-Liste + Schleife in Ihrem Code. Die Kontrolle liegt immer bei der Schleife, nie beim Modell.

Die Schleife im Code: SortingAgent (Paket application)

```
public class SortingAgent {
  private final LlmClient llm;          // Port
  private final ToolDispatcher tools;
  private static final List<ToolSpec> TOOLS = List.of(                    1
    new ToolSpec("listPhotos",
      "Listet die Fotos eines Albums mit Datum",
      "{albumId: string}"),
    new ToolSpec("movePhoto",
      "Verschiebt ein Foto in ein anderes Album",
      "{photoId: string, targetAlbumId: string}"));
  public String run(String task) {
    var history = new ArrayList<Message>();
    history.add(Message.user(task));
    for (int step = 0; step < 20; step++) {                               2
      LlmResponse r = llm.ask(history, TOOLS);                            3
      if (!r.hasToolCall()) return r.text();                              4
      String result = tools.dispatch(r.toolCall());                       5
      history.add(Message.toolResult(result));
    }
    throw new AgentDidNotFinishException();
  }
}
```

1 Tool-Liste

Speisekarte für das LLM: Name, Beschreibung, Parameter. Nur was hier steht, kann es sich wünschen.

2 Schleife mit Limit

Eine Runde = ein LLM-Aufruf + höchstens ein Tool. 20 Runden = Notbremse gegen Endlosschleifen und Kosten.

3 Einziger KI-Aufruf

LlmClient ist ein Port; der Adapter (HTTP zum Anbieter) liegt in infrastructure.

4 Kein Tool-Wunsch = fertig

Niemand schreibt „done“: Das Modell antwortet einfach mit Text.

5 Unser Code führt aus

Das Ergebnis kommt in den Verlauf – so „sieht“ das LLM es in der nächsten Runde.

Die Schleife gehört Ihnen: Was angeboten, ausgeführt und wann abgebrochen wird, entscheidet Java-Code – nicht das LLM.

Ein Lauf, Runde für Runde: „Sortiere ‚Unsortiert' nach Monat"

Runde	LLM antwortet	Unser Code tut	Zurück in den Verlauf
0	`listPhotos("unsortiert")`	ListPhotosUseCase ausführen	`[p1 Jan, p2 Jan, p3 Feb]`
1	`movePhoto("p1", "januar")`	MovePhotoUseCase → Album prüft Regeln	`"ok"`
2	`movePhoto("p2", "januar")`	wie Runde 1	`"ok"`
3	`movePhoto("p3", "februar")`	Album „februar" hat schon 100 Fotos → Exception	`"error: Album voll"`
4	Text: „p1 und p2 sind im Januar. p3 nicht verschoben – Album Februar ist voll."	kein Tool-Wunsch → Schleife endet	–

Wer bestimmt die Reihenfolge? Das LLM – aus Aufgabe, Tool-Beschreibungen und bisherigen Ergebnissen.

Wer setzt die Regeln durch? Ihr Code – die 100er-Regel greift, egal ob die Nutzerin oder der Agent aufruft.

Fehler sind Informationen: Die Fehlermeldung geht an das LLM zurück, und es passt seinen nächsten Schritt an.

Der ToolDispatcher: die Allow-List in Java (Paket application)

```
public class ToolDispatcher {
  private final ListPhotosUseCase listPhotos;
  private final MovePhotoUseCase movePhoto;
  // bewusst KEIN DeleteAlbumUseCase                                    1
  public String dispatch(ToolCall call) {
    return switch (call.name()) {
      case "listPhotos" -> json(listPhotos.execute(
                             AlbumId.of(call.arg("albumId"))));
      case "movePhoto" -> {
        var photo  = PhotoId.of(call.arg("photoId"));                   3
        var target = AlbumId.of(call.arg("targetAlbumId"));
        try {
          movePhoto.execute(photo, target);                             4
          yield "ok";                                                   5
        } catch (DomainException e) {
          yield "error: " + e.getMessage();
        }
      }
      default -> throw new UnknownToolException(call.name());           2
    };
  }
}
```

1 Nur Injiziertes ist erreichbar

Was nicht im Konstruktor steht, existiert für den Agenten nicht.

2 Jeder case = ein erlaubtes Tool

default lehnt alles andere ab. Das ist die Allow-List.

3 Argumente prüfen

PhotoId.of(…) prüft das Format – das LLM kann sich vertippen.

4 Domänenregeln gelten

Der Use Case ruft Album auf: 100er-Regel auch für den Agenten.

5 yield (Java 14+)

Liefert den Wert eines case-Blocks { … } im switch-Ausdruck. Wie return – aber für den switch, nicht die Methode.

Prompts sind Wünsche, Tool-Sets sind Grenzen: Was der Agent nicht darf, steht nicht im Prompt – es fehlt im switch.

Welches Tool wählt das LLM? Beeinflussen – und begrenzen

Beeinflussen

macht eine gute Wahl wahrscheinlich

- **Aufgabe:** „nach Monat sortieren“ macht listPhotos + movePhoto naheliegend
- **Tool-Beschreibung:** der stärkste Hebel – wie Methodenname + Javadoc
- **Systemprompt:** Vorgehen vorgeben („erst auflisten, dann verschieben, IDs nie raten“)
- **Verlauf:** Ergebnisse und Fehlermeldungen lenken den nächsten Schritt

Begrenzen

macht eine falsche Wahl harmlos

- **Allow-List:** nur angebotene Tools existieren
- **Validierung + Use Case:** falsche Argumente und Regelverstöße werden abgelehnt
- **Freigabe:** Irreversibles erst nach Klick der Nutzerin
- **Limit + Protokoll:** kein Endlos-Lauf, jeder Aufruf nachvollziehbar

Schwache Tool-Beschreibung

```
new ToolSpec("move", "moves it",
             "{a: string, b: string}")
```

Starke Tool-Beschreibung

```
new ToolSpec("movePhoto",
  "Verschiebt ein Foto in ein anderes Album. "
  + "Scheitert, wenn das Ziel 100 Fotos hat.",
  "{photoId: string, targetAlbumId: string}")
```

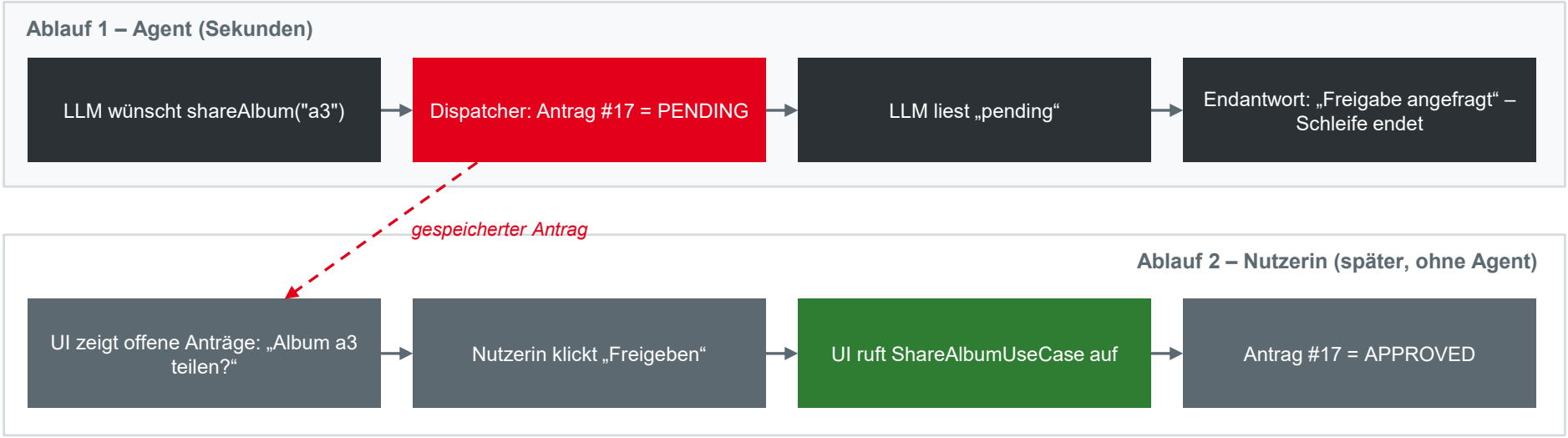
LLM-Antworten sind wahrscheinlich, nicht garantiert. Entwerfen Sie so, dass eine falsche Wahl keinen Schaden anrichtet.

Freigabe durch die Nutzerin: zwei getrennte Abläufe, kein Warten

```
// im ToolDispatcher
case "shareAlbum" -> {
  var id = approvals.request("shareAlbum", call.args());
  yield "pending: wartet auf Freigabe (" + id + ")";
}
```

Das Tool teilt nicht. Es legt nur einen Antrag in der ApprovalQueue ab und meldet dem LLM „pending“.

yield wartet nicht. Es liefert sofort den Text an die Schleife zurück.



Der Agent bekommt nie die Fähigkeit zu teilen. Ausgeführt wird nur, was ein Mensch in der UI auslöst.

Guardrails: Welche Sicherung steht wo im Code?

Guardrail	Wo im Code	Im Photo Album
Kleine Tool-Liste	`TOOLS` in SortingAgent	nur listPhotos, movePhoto – kein generisches executeSql
Allow-List	`switch` im ToolDispatcher, `default` wirft	kein deleteAlbum-case → Löschen unmöglich
Argumente prüfen	`PhotoId.of(…)`, `AlbumId.of(…)`	erfundene ID → Fehlermeldung an das LLM
Domänenregeln	Use Case → Album	100er-Regel, keine Duplikate – für jeden Aufrufer
Ausgabe prüfen	LLM-Adapter (z. B. DescriptionSuggester)	Vorschlag ≤ 100 Zeichen, keine URLs
Freigabe	ApprovalQueue + UI	shareAlbum erst nach Klick der Nutzerin
Limit + Protokoll	`step < 20`, Log je Tool-Aufruf	Abbruch nach 20 Runden; jeder Aufruf nachvollziehbar

Jede Zeile ist eine Klasse oder eine Code-Zeile, die man testen kann. Das ist der Unterschied zu „Guardrails im Prompt“.

Wo liegt das in Ihrer Architektur? Der Agent ist nur ein weiterer Client

ui	**PhotoAlbumView** (Nutzerin) · **ApprovalView** (offene Anträge)
application	**Use Cases:** AddPhoto · MovePhoto · ShareAlbum … **Agent (neu):** SortingAgent · ToolDispatcher · ApprovalQueue **Ports:** AlbumRepository · LlmClient · DescriptionSuggester
domain	**Album, Photo** – alle Geschäftsregeln, kennt keine KI
infrastructure	**FileAlbumRepository** · **ClaudeLlmClient / OpenAiLlmClient** (Adapter)

Begriffe, die Sie hören werden

MCP

Model Context Protocol: offener Standard für genau das, was ToolSpec und dispatch tun – nur über Prozessgrenzen hinweg.

Skills

versionierte Anweisungsdateien (Systemprompt-Bausteine), die ein Agent bei Bedarf lädt.

Konzept bleibt gleich: Schleife, Tool-Liste, Dispatcher, Guardrails.

Der Agent ruft dieselben Use Cases auf wie die UI – mit weniger Rechten als die Nutzerin. Die Domäne kennt keine KI.

Übung: KI-Funktion und Agent in Ihrer Architektur

KI-Perspektive

Ihre Projekt-KI-Funktion „Beschreibung vorschlagen“ ist eine einzelne LLM-Anfrage – kein Agent. Entwerfen Sie zuerst sie, dann einen Agenten.

10 min

Team

AUFTRAG

1. Zeichnen Sie DescriptionSuggester als Port in die Anwendung und einen Adapter in die Infrastruktur. Welche zwei Prüfungen macht der Adapter (vor der Anfrage, nach der Antwort)?

2. Agent „Fotos aufräumen“: Schreiben Sie die ToolSpec-Liste (höchstens vier Tools, je eine Beschreibung). Ordnen Sie zu: direkt ausführen · nur mit Freigabe · gar nicht anbieten – für deleteAlbum, removePhoto, renameAlbum, shareAlbum.

3. Entscheiden Sie: Bleibt die KI-Funktion Won't (dokumentiert) oder Should (mit FakeSuggester in Tests)?

ERGEBNIS Ergänzung im Klassendiagramm + fünf Zeilen „KI-Komponente“ in ARCHITECTURE.md.

DANACH IM PLENUM Zwei Teams zeigen ihre Tool-Liste. Prüffrage: Steht eine Grenze im Prompt statt im switch?

Architecture Decision Record (ADR): zehn Zeilen, die Punkte bringen

- **Titel und Status**
  - ADR-001: Speicher hinter AlbumRepository-Interface · Status: akzeptiert · Datum
- **Kontext**
  - Welche Kraft wirkt? NFA Testbarkeit; Dateispeicher ist gefordert, Datenbank denkbar
- **Entscheidung**
  - Port AlbumRepository in application; FileAlbumRepository in infrastructure; InMemory in Tests
- **Alternativen (verworfen, mit Grund)**
  - direkter Dateizugriff im Use Case (nicht testbar); Factory (unnötig bei einer Konfiguration)
- **Konsequenzen**
  - + Unit-Tests ohne Dateisystem, + Speicher austauschbar; – ein Interface mehr, Mapping-Code

Ort: docs/adr/ADR-001-repository.md. Mindestens ein ADR ist Pflicht; drei bis vier sind realistisch (Speicher, Fehlerbehandlung, UI-Technik, KI-Port).

Architecture Studio

Teamarbeit

Aus CRC-Karten, Sequenz und Pattern-Entscheidungen wird Ihr Entwurf. Arbeitsteilung im Team: zwei Personen Diagramme, zwei Personen ARCHITECTURE.md und ADR.

25 min

Team

AUFTRAG

1. Komponentenübersicht: vier Schichten, welche Klassen wo, Abhängigkeitsrichtung (Skizze reicht).
2. Klassendiagramm (Domäne + Anwendung + Ports) aus den CRC-Karten – Multiplizitäten, nur entscheidungsrelevante Methoden.
3. Sequenzdiagramm „Foto hinzufügen“ inklusive Fehlerpfad (Duplikat) als alt-Fragment; Konsistenz mit dem Klassendiagramm prüfen.
4. Ein ADR zu Ihrer wichtigsten Entscheidung (Speicher, Fehlerbehandlung, UI-Technik oder Pattern).
5. Commit „docs: architecture v1“ – Diagramme als PlantUML oder Bild in docs/design/.

ERGEBNIS ARCHITECTURE.md v1 mit Verweisen auf Diagramme und ADR im Repository.

Diagramm-Review über Kreuz

Teamarbeit

Zuordnung wie in Block 2 (1->2->3->4->5->1). Sie reviewen Klassendiagramm und Sequenzdiagramm des Partnerteams.

10 min

Teams über Kreuz

AUFTRAG

1. Regel-Check (3 min): Finden Sie die Klasse, in der jede Geschäftsregel geprüft wird. Fehlt eine? Ist eine in der UI?
2. Konsistenz-Check (3 min): Gibt es eine Nachricht im Sequenzdiagramm ohne passende Operation im Klassendiagramm?
3. Kopplungs-Check (2 min): Welche Klasse hat die meisten Abhängigkeiten? Ist das begründet?
4. Issue „review: architecture v1“ mit drei Punkten im Partner-Repository (2 min).

ERGEBNIS Ein Review-Issue erhalten, eines vergeben. Einarbeitung ist Hausaufgabe.

Projektcheckpoint Block 3

Teamarbeit

Jedes Team bewertet sich selbst: grün = im Repository sichtbar · gelb = begonnen · rot = fehlt. Für jeden gelben/roten Punkt entsteht ein Issue mit Verantwortlichem.

- ARCHITECTURE.md: Schichten/Komponenten mit Verantwortlichkeiten und Begründung
- Relevante Schnittstellen (Ports) benannt: Repository, Hasher/Storage, ggf. KI
- Klassendiagramm konsistent mit CRC-Karten; Geschäftsregeln in der Domäne
- Sequenzdiagramm „Foto hinzufügen“ inkl. Fehlerpfad, konsistent mit Klassendiagramm
- Mindestens ein ADR mit verworfener Alternative
- Pattern nur mit benanntem Problem (oder bewusst keins)
- Review-Issue erhalten; AI_USAGE.md Fall 1 aus Block 2 vorhanden

Abgabe des Zwischenstands: Commit/Tag im Repository bis zum Vorabend des nächsten Blocks.

Exit Ticket und bis zum nächsten Block

Exit Ticket (3 Minuten, jede Person schriftlich)

- Unsere wichtigste Entwurfsentscheidung lautet … Sie schützt uns vor … Der Preis dafür ist …
- Welche Verantwortung haben wir heute verschoben – und warum?
- Welches Pattern haben wir bewusst NICHT eingesetzt?

Bis zum nächsten Block (Teamarbeit)

- Review-Issue einarbeiten; ARCHITECTURE.md v1 finalisieren (Tag „block-3“)
- Projektgerüst anlegen: Maven/Gradle-Projekt mit Paketstruktur domain/application/infrastructure/ui, Album und Photo als leere Klassen mit Konstruktoren und Invarianten
- GitHub-Repository: alle Teammitglieder mit Zugriff; Branch-Schutz für main aktivieren (PR erforderlich)
- Jede Person: Java 21 + IDE + Git lokal lauffähig; `mvn verify` bzw. `gradle build` läuft grün (auch ohne Tests)
- AI_USAGE.md Fall 2: KI ein Klassendiagramm-Review machen lassen; einen Vorschlag übernehmen, einen verwerfen – mit Begründung

Quellen und Materialbasis Block 3

- Vorwissen: SAE Einheit 4 (Klassendiagramm), 01_Foundations.pptx (Kapselung, ADT), 02_Inheritance.pptx (Use vs. inherit, Substitution)
- 05_Domain_Modeling.pptx und 05a_Domain_Modeling_Example.pptx – Nomen-Analyse, CRC-Karten
- 12_Design_Patterns_long.pptx; Gamma, Helm, Johnson, Vlissides: Design Patterns (1994)
- Larman, C.: Applying UML and Patterns – GRASP (Information Expert, Controller, Protected Variations)
- Seidl et al.: UML@Classroom (Sequenzdiagramme); Rupp: UML 2 glasklar
- Nygard, M.: Documenting Architecture Decisions; Martin, R.: Clean Architecture (Abhängigkeitsregel)
- OWASP Top 10 for LLM Applications; Model Context Protocol – Security Best Practices; Anthropic: Building effective agents
- Cockburn, A.: Hexagonal Architecture (Ports and Adapters), 2005; Martin, R. C.: Clean Architecture, 2017
- Java Language Specification / JEP 361: Switch Expressions (yield), Java 14